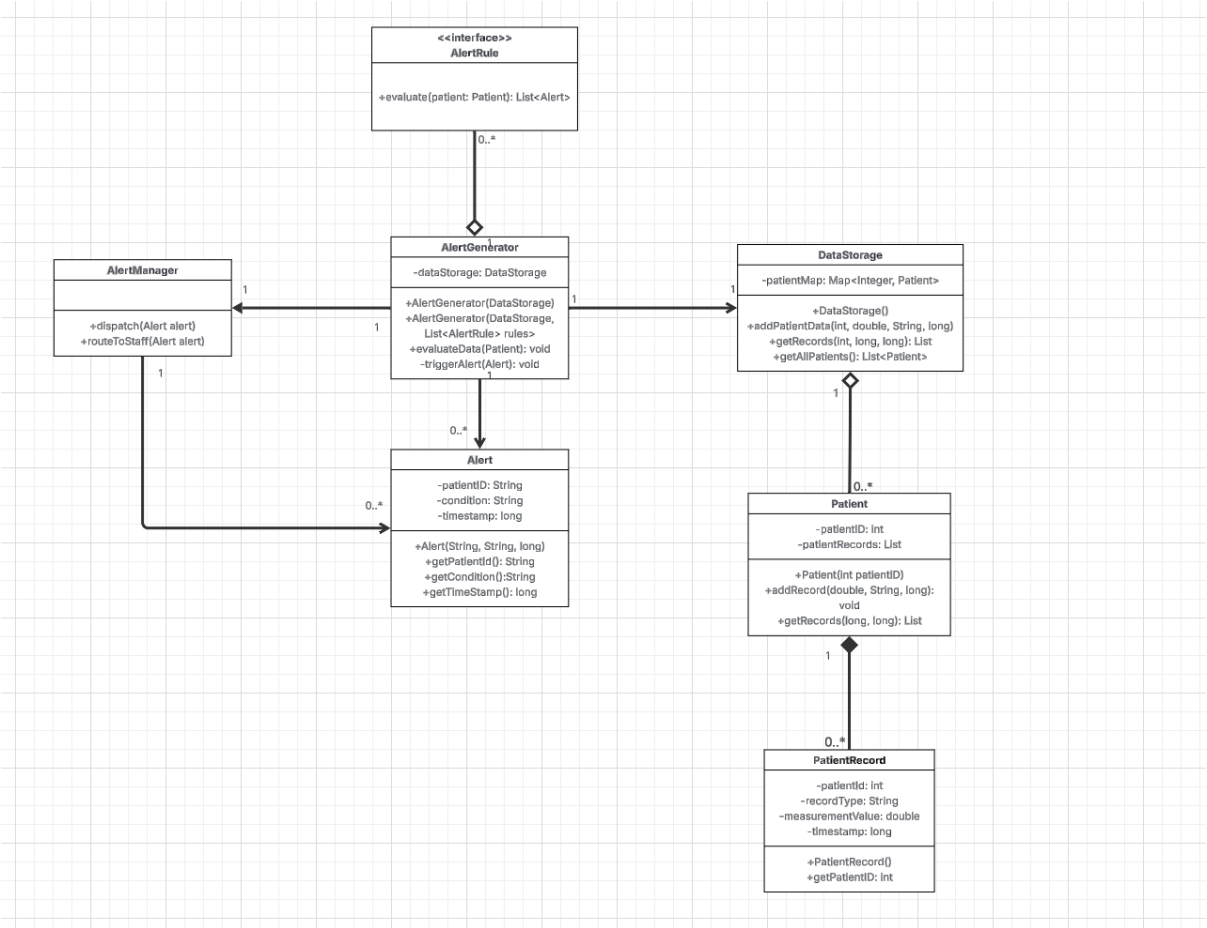
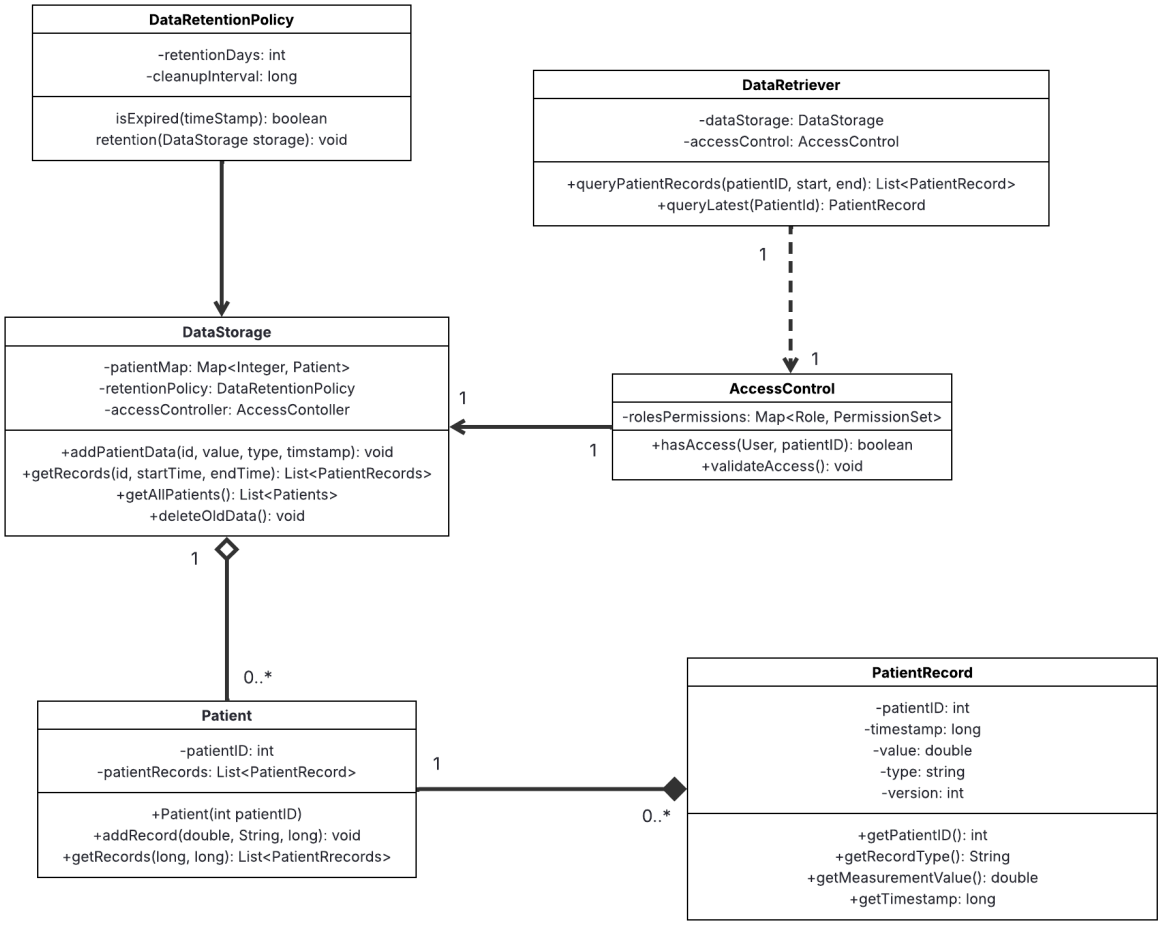
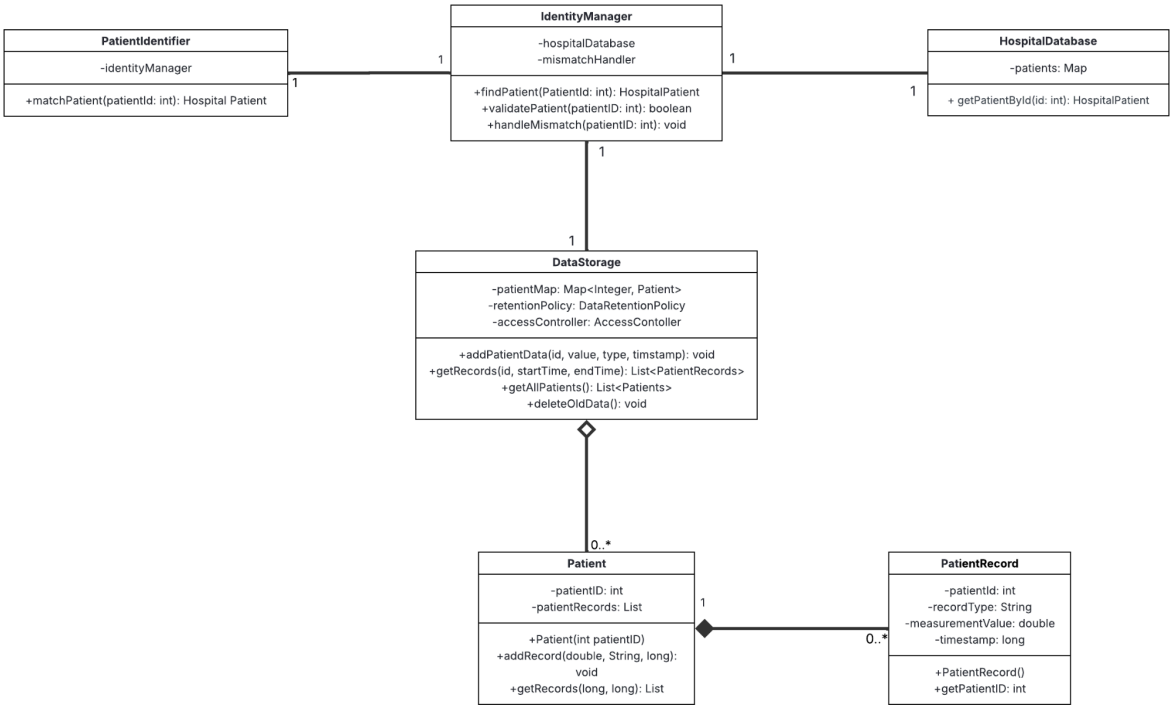


Rationale:

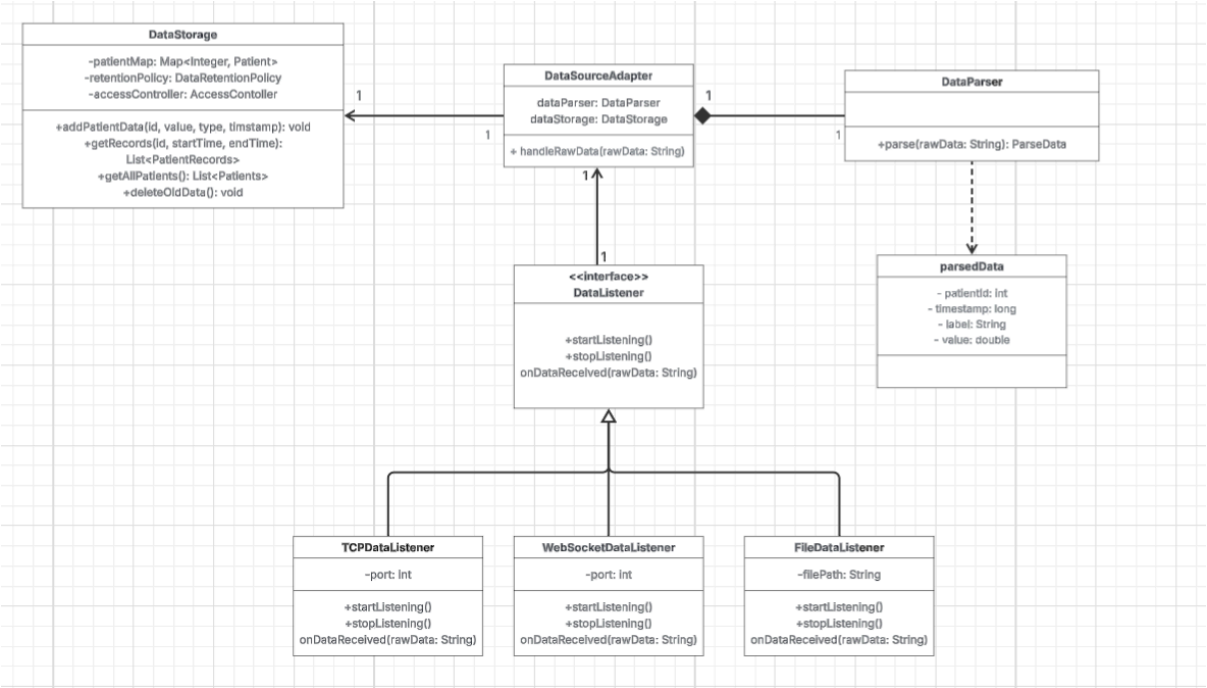
This UML class diagram is for the alert generation system. The DataStorage class has a one-to-many relationship(1 to 0..*) with Patient because this one storage system keeps many patients inside a Map. Each patient belongs to that one storage system, hence composition is used. If storage is removed then access to patients is lost. Each Patient has a one to many relationship with PatientRecord. This is also composition because a record cannot exist without a patient. AlertGenerator has a one to one relationship with DataStorage because it uses the storage to read patient data, it does not own the data just accesses it, so it is only an association and not composition. The relationship between AlertGenerator and Patient is 1 to 0..*, as one generator can check multiple patients, via data storage, so it can have indirect dependency. Patients are passed into the method and not stored inside the class.



Rationale:
DataStorage acts as the main repository, containing all the patient information. All data passes through this one point before being retrieved. Each Patient is linked to DataStorage using a 1 to many aggregation relationship, each patient still exists even if the storage is reorganized. Hence aggregation because it means the storage “contains” but doesn't strictly own patients. Inside each Patient, there is a 1 to many composition relationship with PatientRecord. PatientRecord cannot exist without a patient. If a patient is deleted, all their records must also be deleted, hence the composition. DataRetriever is responsible for querying data, not storing it. Its dependency relationship with DataStorage reflects that it uses storage but does not own or modify it directly. It also depends on AccessControl, making sure that every query is checked before returning data. The 1-to-1 relationship is used because one retriever execution operates with one access control operation. AccessControl is connected to DataStorage with a 1-to-1 relationship because a single storage system enforces a single security layer. This simplifies the permission process and equally applies to all patients.



Composition is used between DataStorage and Patient, and between Patient and PatientRecord. This is because these objects have a strong ownership relationship. A Patient cannot exist in the system without being stored in DataStorage, and a PatientRecord cannot exist without a Patient. If the DataStorage is deleted, all Patient objects are lost, and if a Patient is removed, all their records are also removed. The multiplicity is 1 to 0..* because one DataStorage can contain many patients, and one Patient can contain many records. Aggregation is used between HospitalDatabase and HospitalPatient. This is a weaker relationship. The database stores hospital patients, but those patients represent real-world entities that can exist independently of the system. Even if the database is reset, the concept of a hospital patient still exists. Therefore, aggregation is appropriate. The multiplicity is 1 to 0..* because one database manages many hospital patients. Association is used between PatientIdentifier, IdentityManager, DataStorage, and HospitalDatabase. These classes interact but do not own each other. For example, PatientIdentifier uses IdentityManager to match patients, and IdentityManager queries both DataStorage and HospitalDatabase. These are functional relationships, not ownership. The multiplicity is mostly 1 to 1 because each component relies on a single instance of the other to perform its task.



Rationale:

This UML diagram is designed around how data flows from external sources into the system and then gets transformed before being stored. The DataListener uses inheritance because TCPDataListener, WebSocketDataListener, and FileDataListener are all specific ways of receiving data but share the same behavior contract. This is a “is-a” relationship. The relationship from DataListener to DataSourceAdapter is an association because the listener needs to notify or forward incoming raw data to the adapter, but it does not own or control its lifecycle; it simply holds a reference and uses it when events occur. The multiplicity is 1 to 1 because each listener instance typically forwards data to a single processing pipeline (adapter), ensuring a clear and predictable flow per source. The DataSourceAdapter to DataParser relationship is composition because the adapter fully controls parsing behavior and the parser is not meant to exist independently in this design; if the adapter is destroyed, the parser is useless. The multiplicity is 1 to 1 because each adapter uses exactly one parsing strategy instance at a time to ensure good interpretation of data. The adapter to DataStorage relationship is an association because storage is a shared component used across the system and is not owned by the adapter. Its multiplicity is 1 to 1 because each adapter instance writes to a single central storage unit. Finally, DataParser and ParsedData have a dependency relationship because the parser only creates and returns parsed objects temporarily without storing them, so no ownership exists. The adapter also depends on ParsedData in the same way, as it only uses the object to pass structured data into storage.